

AGENT-NATIVE DATA INFRASTRUCTURE

Own the data, *not* *the dashboard.*

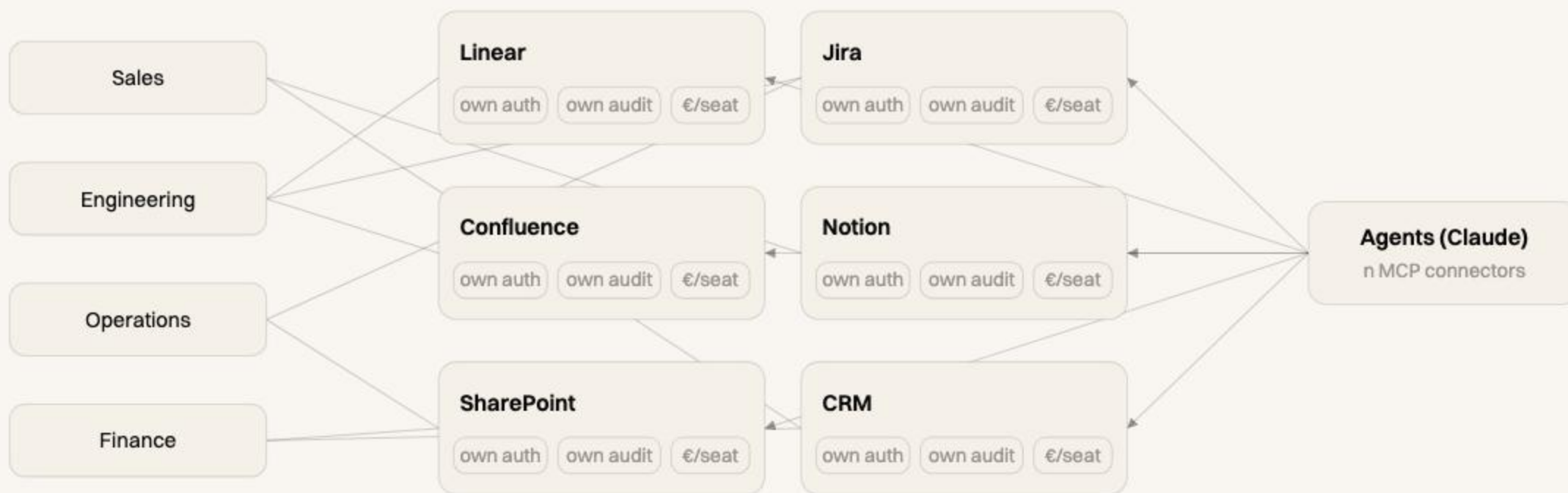
Your agents do the work now. Temple replaces the SaaS interfaces nobody opens: one owned data layer, one permission model, one audit trail.

● live demo · `temple-incredicorp-production.up.railway.app`

01

Every tool you stopped opening
is just a database wearing a UI.

Six databases wearing six interfaces.



6 subscriptions · 6 permission models · 6 audit trails · 0 owned

Not “is it a database underneath”.
It is “does anyone other than
an agent need the surface”.

Collapses to schema

- Project tracking, Linear and Jira
- Docs and wiki, Notion, Confluence, SharePoint
- CRM and large parts of ERP

Pure control-logic-over-a-database. Drop the subscription, keep the data.

Worth paying for

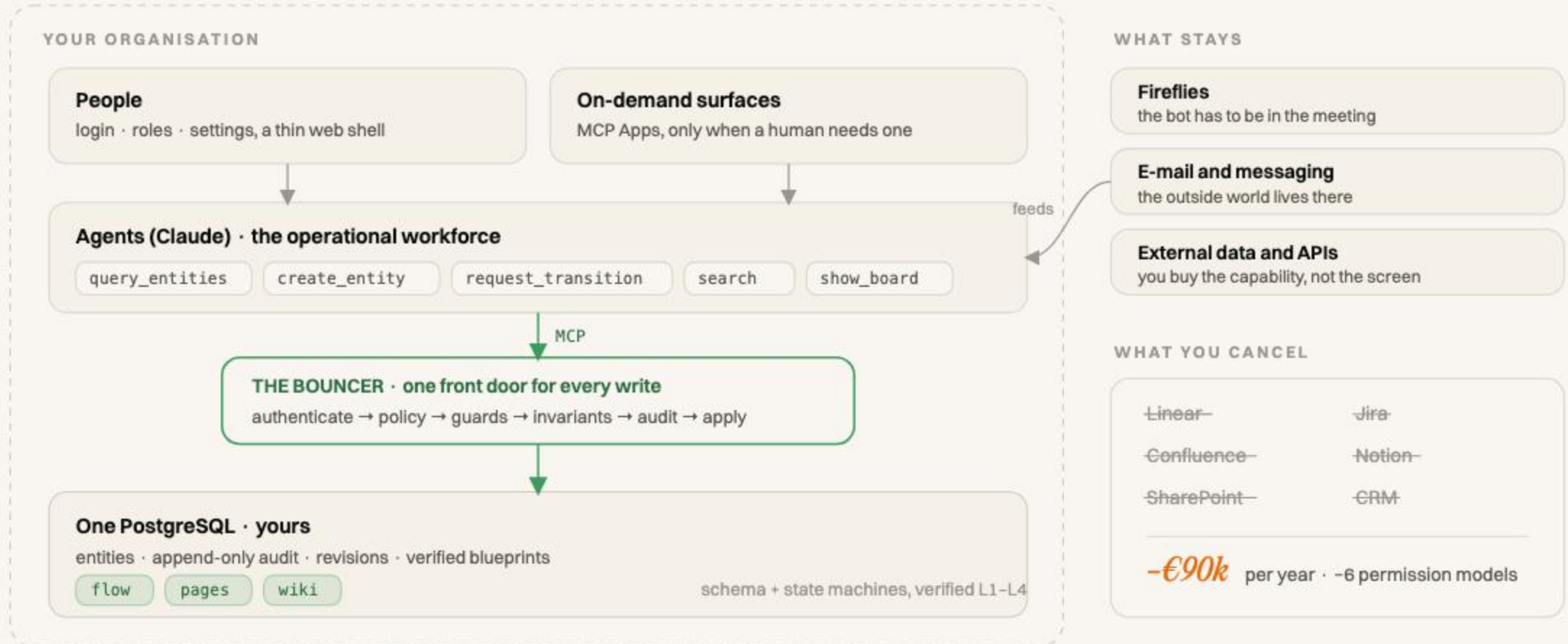
- Capture, where a bot must attend, Fireflies
- External data and enrichment networks
- The rare human moment, rendered on demand

You buy capability, not an interface.

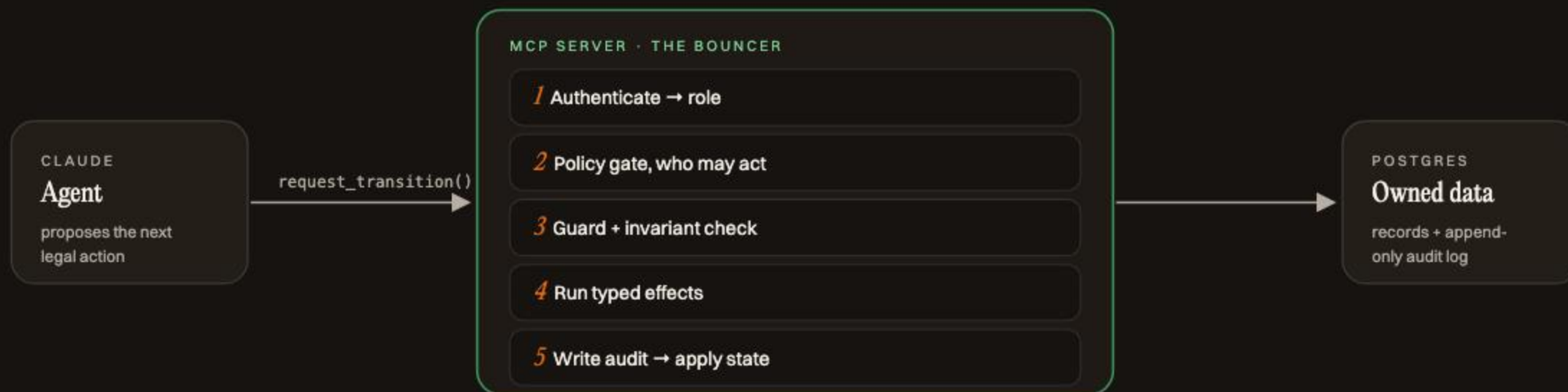
02

One owned data
layer. Everything
else is agents talking.

Where Temple sits in your organisation.



Every write goes through the bouncer. No exceptions, no side doors.



The same pipeline serves the web shell and every agent. A viewer cannot transition, an agent cannot approve, and every change lands in the audit log.

One engine. Three shapes of work.

A blueprint is declarative data, not code: states, legal transitions, guards, invariants, role policy. The engine runs whatever shape you declare.

Flow

linear-like · state machine

Issues move through legal transitions only.
Guards block half-done work, invariants make bad states unreachable, policy says who may approve.

Pages

notion-like · typed databases

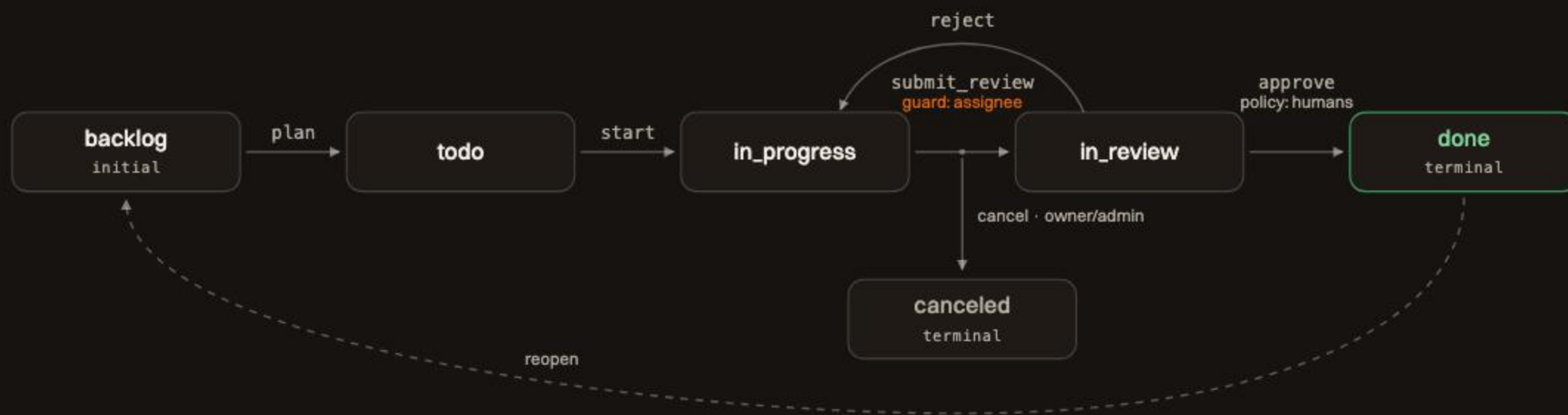
Databases hold pages with typed properties and a markdown body. Loosely structured, fully governed, every edit versioned.

Wiki

free-form · revisions + backlinks

The loosest shape: markdown pages in a tree.
An immutable revision on every save, [[backlinks]] extracted automatically.

A blueprint is a verified state machine.



This is `blueprints/flow.json`, exactly as deployed. An agent can start work and submit it for review. Only a human can approve, cancel or reopen.

“Verified” is a gate, not a sticker.

L1

Algebra

One initial state, every transition references real states.

L2

Reachability

No unreachable states, no dead ends, terminals stay terminal.

L3

Soundness

No path of legal moves can reach a state that violates an invariant.

L4

Property

200 randomised action sequences, invariants hold throughout.

```
$ npm run verify
```

```
PASS blueprint: flow      L1 algebra ok · L2 reachability ok · L3 soundness ok · L4 property ok
```

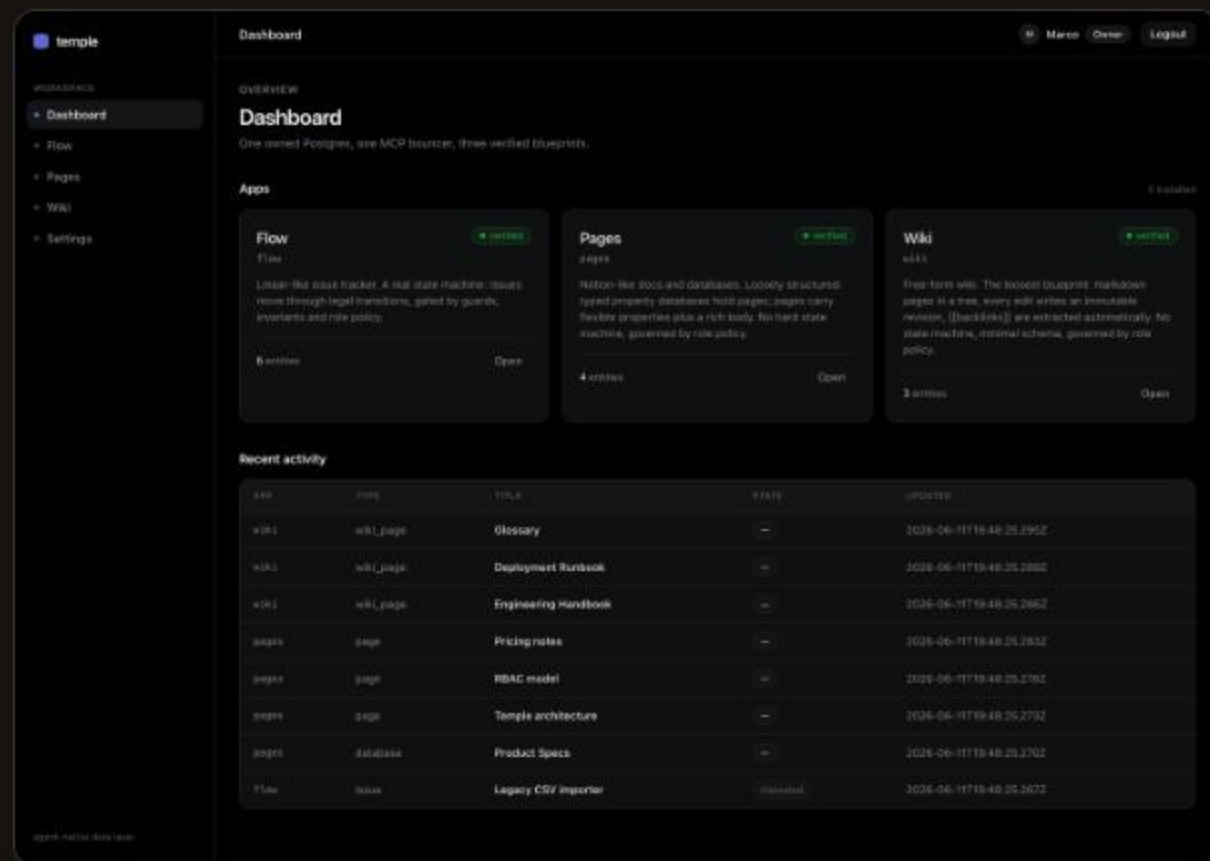
```
PASS blueprint: pages    L1 ok · L2 ok · L3 ok · L4 ok
```

```
PASS blueprint: wiki     L1 ok · L2 ok · L3 ok · L4 ok
```

A blueprint that fails any level does not load. Change the spec, the verifier re-proves it before the engine will run it.

PROOF

Built and deployed, not a concept.



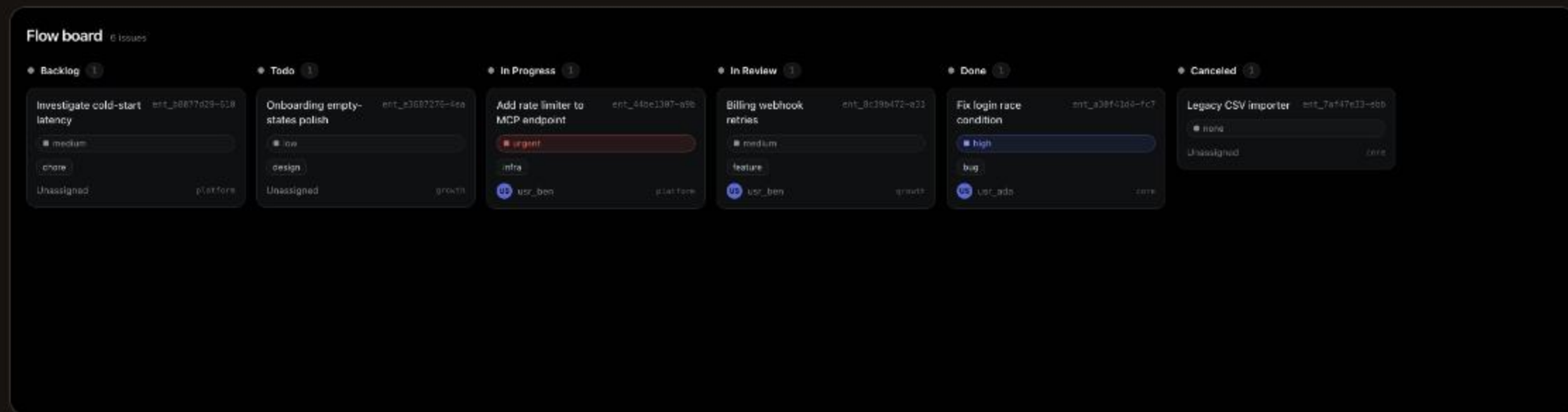
The running product

- Three verified apps over one PostgreSQL
- Login, roles and settings, the only human chrome
- 16/16 engine tests green, 3/3 blueprints verified
- Live on Railway with the seeded demo workspace

temple-incredicorp-production
.up.railway.app

AGENTS GET A SURFACE TOO

MCP Apps render the work inside Claude.



The Flow board as an MCP App: ask Claude to show the board and this renders inline, live from the engine. No tab switch, no second login, same permissions.

When a person does need a screen, it appears, then it leaves.

Flow / Issues / [ent_8c30b472-a31](#)

Billing webhook retries

In Review

PROPERTIES

Description	—
Assignee	usr_ben
Priority	medium
Estimate	2
Labels	feature
Team	growth

AUDIT TRAIL

create	← backlog
Ben Carter · 2026-06-11T19:48:25.256Z	
start backlog	→ in progress
Ben Carter · 2026-06-11T19:48:25.258Z	
submit_review	in progress → in review
Ben Carter · 2026-06-11T19:48:25.260Z	

Governance you can see

- The record card shows only the actions this caller's role is allowed to take
- The audit trail underneath is the real append-only log: who moved what, from which state, when
- Rendered as an MCP App, on demand, inside the conversation

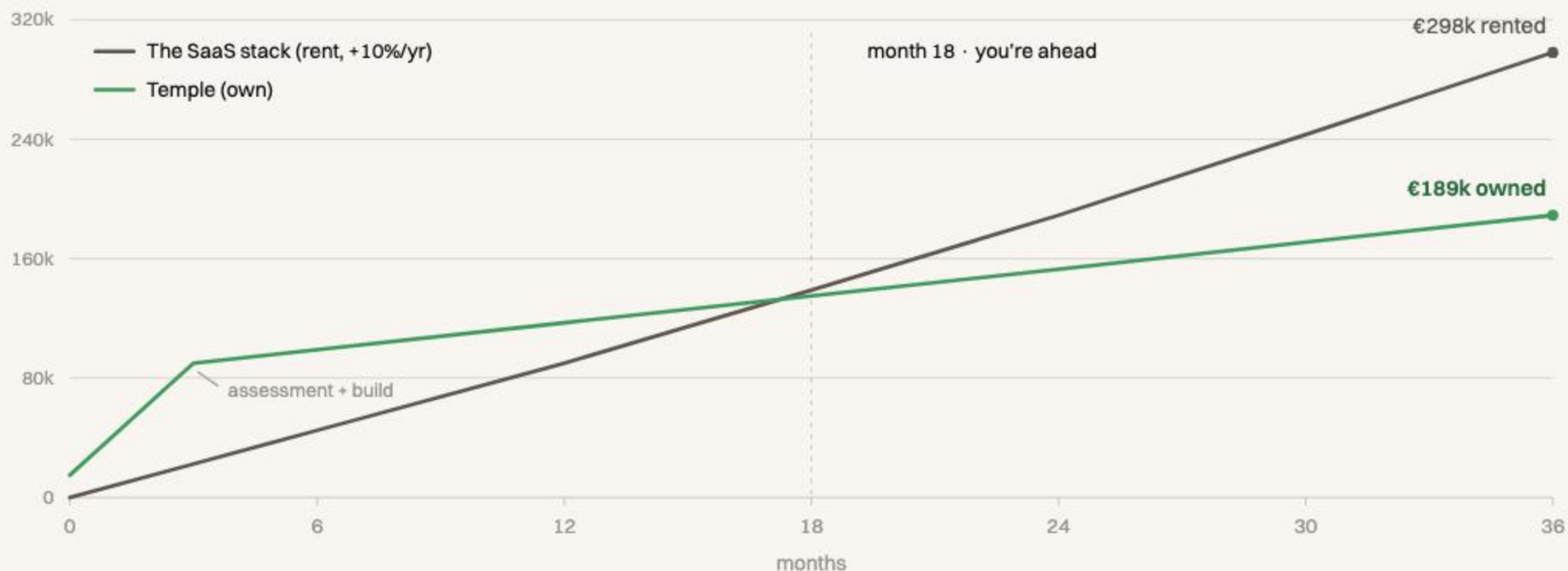
03

Same work. One layer. Less
than half the run-rate.

The stack versus the layer.

	THE SAAS STACK	TEMPLE
Cost model	Per seat, per tool, forever	✓ Build once, flat run-rate
Permissions	One model per vendor, six to govern	✓ One policy layer, enforced in the bouncer
Audit trail	Scattered, partial, vendor-shaped	✓ Append-only, every transition, one place
Agent access	n connectors over UI-shaped APIs	✓ Native: MCP is the front door
Workflow logic	Implicit in screens and habits	✓ Explicit state machines, verified L1-L4
Your data	In six vendor silos	✓ In your PostgreSQL
Exit cost	A migration project per tool	✓ Zero. It is already yours.

Renting compounds. Owning flattens.



A €90k/yr stack with ~10% yearly price-and-seat creep: crossover at month 18, €109k kept by month 36, steady-state under half the run-rate, and you own the asset.
Incredicorp

One lifecycle at a time, never a big bang.

01

Educate

Show which subscriptions are pure interface, and what owning the data layer changes.

02

Prioritise

Map one lifecycle to an explicit, verified blueprint. Fixed-fee assessment.

03

Build

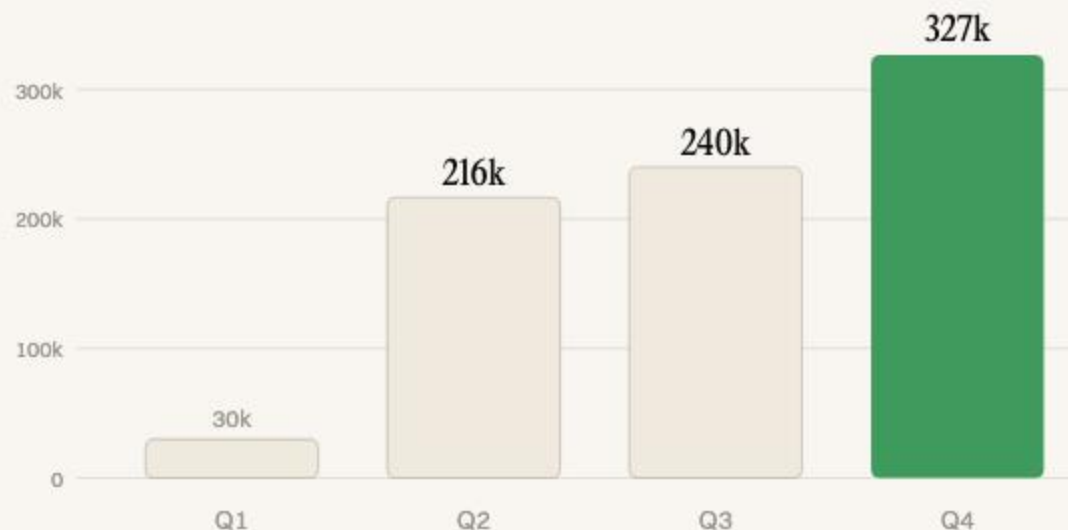
Stand up the owned layer and MCP bounce. Migrate that one workflow off SaaS.

04

Run

Host, maintain, govern. Cancel the subscription it replaced. Repeat.

A new revenue line, built on existing motion.



LINE	CONS.	BASE	UP
Assessments	90k	150k	210k
Implementations	300k	525k	750k
Managed run	45k	90k	150k
Blueprint library	24k	48k	60k
Year 1	€459k	€813k	€1.17M

Illustrative. ~60% blended gross margin. Exit ARR into year two: 6 managed-run clients × €36k plus 8 blueprint packs × €6k = €264k.

WHY NOW

Datadog just proved the engine on stage.

On Anthropic's conference stage, Datadog's VP of Engineering, Sesh Nalla, presented Tempor: a machine tool where agents emit a verified state-machine spec instead of improvised code. Same first principle, reached independently: most software is control logic around a database. They built it as internal infrastructure. The productised, governed version for the mid-market is wide open.

They proved the engine

Declarative blueprints, a compiler outside the LLM, a multi-level verifier, hot-reload into a hardened runtime, running against production-scale workloads.

We own the wedge

GDPR-ready, audit-clean, sold one lifecycle at a time to organisations that will never build this themselves.

THE CLOSE

Stop renting interfaces you no longer open.

We never sell a vision we can't build. This one is already running, one verified lifecycle at a time.

• `temple-incredicorp-production.up.railway.app` • sign in: `marco@incredicorp.dev` / temple